

# INDUSTRY PROBLEM SOLVING TASK

## 1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

### Introduction

Cache memory is a small and high-speed memory located close to the CPU. It stores frequently accessed instructions and data so that the processor does not need to access slower main memory every time. Modern processors generally use multiple levels of cache such as **L1, L2 and L3**.

The basic memory hierarchy is:

**CPU → L1 Cache → L2 Cache → L3 Cache → DRAM → Secondary Storage**

As we move away from the CPU, capacity increases but access speed generally decreases.

## Comparison of L1, L2 and L3 Cache

| Parameter         | L1 Cache                | L2 Cache        | L3 Cache                |
|-------------------|-------------------------|-----------------|-------------------------|
| Size              | Small                   | Medium          | Large                   |
| Location          | Closest to CPU core     | Near CPU core   | Usually shared          |
| Latency           | Lowest                  | Low             | Higher than L1/L2       |
| Bandwidth         | Very high               | High            | High                    |
| Throughput        | Very high               | High            | High                    |
| Cost per bit      | Highest                 | High            | Lower                   |
| Access speed      | Fastest                 | Fast            | Relatively slower       |
| Typical use       | Immediate CPU data      | Backup for L1   | Shared data among cores |
| Power consumption | Low capacity, low total | Moderate        | Higher due to size      |
| Sharing           | Usually private         | Usually private | Often shared            |

### L1 Cache

L1 is the **fastest and smallest cache**. It is normally divided into:

- **Instruction Cache (L1-I)**
- **Data Cache (L1-D)**

The CPU checks L1 first because it has the lowest latency.

### Advantages

- Extremely fast access
- High bandwidth
- Reduces CPU waiting time

- Improves instruction execution

## Limitation

Its small size means it cannot store a large amount of data.

## L2 Cache

L2 is larger than L1 but slower. It acts as a second-level cache when data is not found in L1.

## Advantages

- Larger capacity than L1
- Reduces DRAM accesses
- Provides good balance between speed and capacity

## L3 Cache

L3 is normally the largest cache and is often shared among multiple CPU cores.

## Advantages

- Large capacity
- Reduces main-memory accesses
- Helps multicore processors share frequently used data
- Improves overall throughput

## Impact on Processor Performance

When the processor requests data:

**L1 Hit → Data obtained quickly**

If L1 misses:

**L1 Miss → L2 Check**

If L2 also misses:

**L2 Miss → L3 Check**

If L3 misses:

**L3 Miss → DRAM Access**

Therefore:

**Higher Cache Hit Rate → Fewer DRAM Accesses → Lower Average Memory Access Time  
→ Higher CPU Performance**

## Example

Suppose a processor repeatedly accesses the same instructions in a loop. If those instructions are stored in L1 cache, the processor can repeatedly access them without going to DRAM.

This saves time and increases instruction throughput.

## Solution

To reduce lag while switching between apps, redesign the smartphone memory hierarchy as follows:

- **Increase L1 Cache** to store frequently accessed instructions and data for faster CPU access.
- **Increase L2 Cache** (e.g., from 512 KB to 1 MB per core) to keep more application data close to the processor.
- **Increase Shared L3 Cache** (e.g., from 4 MB to 8 MB) so multiple cores can quickly access recently used app data.
- **Implement Intelligent Cache Prefetching** to predict and load frequently used applications into cache before they are opened.
- **Upgrade DRAM to LPDDR5X/LPDDR6** for higher bandwidth and lower memory access latency.

## Outcome:

- Reduced app-switching latency
- Higher cache hit rate
- Fewer DRAM accesses
- Improved throughput
- Faster and smoother multitasking performance in smartphones.

## Conclusion

L1, L2 and L3 caches form an important hierarchical memory system. L1 provides the lowest latency, L2 provides greater capacity with low latency, and L3 provides large shared storage. A properly designed cache hierarchy improves CPU utilization, reduces memory stalls and increases system throughput.

## 2. Cloud Server Memory Bottleneck

**A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.**

Problem Understanding

A cloud service provider experiences high latency in database queries. The main causes can be frequent cache misses, slow main-memory access, and excessive virtual-memory paging. A suitable memory hierarchy is required to reduce latency and improve database performance.

Cache Hierarchy vs Virtual Memory Paging

| Feature           | Cache Hierarchy                   | Virtual Memory Paging                  |
|-------------------|-----------------------------------|----------------------------------------|
| Purpose           | Reduces CPU memory-access latency | Extends available memory using storage |
| Main technologies | SRAM-based L1, L2, L3 cache       | DRAM + SSD/HDD                         |
| Speed             | Very high                         | Much slower when page faults occur     |
| Access unit       | Cache line                        | Page                                   |
| Location          | Close to CPU                      | DRAM and secondary storage             |
| Main problem      | Cache miss                        | Page fault                             |
| Best use          | Frequently accessed data          | Data that cannot fit in RAM            |

Analysis

CPU caches store frequently used database instructions and data. L1 cache is the fastest but smallest, while L2 and L3 provide larger storage with slightly higher latency.

Virtual memory allows applications to use more memory than physically available. However, when required pages are not present in DRAM, a page fault causes data to be fetched from SSD/storage. This is several orders of magnitude slower than cache or DRAM access.

Therefore, increasing cache efficiency and avoiding excessive paging are both important.

Recommended Memory Hierarchy

CPU → L1 Cache → L2 Cache → L3 Cache → DRAM → NVMe SSD

Recommended improvements:

- 1. Increase L3 cache capacity for frequently accessed database data.
- 2. Use high-speed DDR5/LPDDR5X-class DRAM where appropriate.
- 3. Increase physical RAM to reduce page faults.
- 4. Use NVMe SSD instead of HDD for swap/storage.
- 5. Optimize database indexes and query caching.
- 6. Use memory-aware database buffer pools.
- 7. Monitor cache-miss rate and page-fault rate.

Solution:

To reduce database query latency, the cloud provider should optimize both the cache hierarchy and virtual memory system.

Cache Hierarchy vs Virtual Memory Paging

- **L1, L2, and L3 Cache:** Very fast memory levels that store frequently used data and instructions close to the CPU, reducing access time.
- **Virtual Memory Paging:** Uses disk space as extra memory when RAM is full. Accessing data from disk is much slower and increases latency.

### Outcome:

- Reduced query latency
- Higher cache hit rate
- Fewer page faults
- Faster database performance
- Improved server throughput and scalability

### Conclusion

Cache hierarchy should be optimized for **low latency**, while sufficient DRAM and fast NVMe storage should be used to minimize costly virtual-memory paging. The best solution is a balanced hierarchy rather than relying heavily on paging.

## 3. Autonomous Vehicle Data Processing

**An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.**

### Problem Understanding

An autonomous vehicle continuously processes camera, LiDAR, radar, GPS, and other sensor data. These workloads require very low latency and high memory bandwidth. A memory hierarchy must provide fast access to frequently used data while maintaining sufficient capacity.

### SRAM vs DRAM vs MRAM

| Feature    | SRAM             | DRAM             | MRAM                   |
|------------|------------------|------------------|------------------------|
| Speed      | Extremely high   | High             | High                   |
| Latency    | Very low         | Higher than SRAM | Low                    |
| Density    | Low              | High             | Medium                 |
| Volatility | Volatile         | Volatile         | Non-volatile           |
| Power      | Higher per bit   | Requires refresh | Low standby power      |
| Cost       | High             | Lower            | Higher                 |
| Best use   | CPU/cache memory | Main memory      | Fast persistent memory |

## Proposed Memory Hierarchy

**Sensor → SRAM Cache → DRAM Main Memory → MRAM Persistent/Fast Storage**

A practical architecture can use:

- **SRAM:** L1/L2/L3 CPU and accelerator caches for immediately required sensor data.
- **DRAM:** Main working memory for camera frames, LiDAR point clouds, maps, and neural-network intermediate data.
- **MRAM:** Non-volatile storage for frequently required parameters, configuration data, selected AI models, and critical state information.

## Design

1. Sensor data enters through high-speed interfaces.
2. Frequently accessed data is placed in SRAM caches.
3. Large sensor datasets are stored in DRAM.
4. Important persistent data can be maintained in MRAM.
5. DMA can transfer sensor data directly to memory without unnecessary CPU intervention.
6. Parallel memory channels can increase bandwidth.
7. Data locality should be exploited to reduce latency.

## Solution:

### Evaluation of Memory Technologies

#### 1. SRAM (Static RAM)

- Fastest memory technology.
- Very low latency.
- Expensive and lower storage capacity.
- Used for CPU caches (L1, L2, L3).

**Advantage:** Extremely fast access for real-time sensor processing.

#### 2. DRAM (Dynamic RAM)

- Larger storage capacity.
- Moderate speed.
- Lower cost than SRAM.
- Used as the main memory.

**Advantage:** Stores large amounts of sensor and map data.

#### 3. MRAM (Magnetoresistive RAM)

- Non-volatile memory.
- Faster than flash memory.
- Retains data even when power is lost.
- Low power consumption.

**Advantage:** Stores critical vehicle data and AI models safely.

#### Outcome:

- Reduced latency in sensor processing
- High bandwidth for camera and LiDAR data
- Faster autonomous decision-making
- Improved reliability and safety
- Lower power consumption

#### Conclusion

SRAM provides the lowest latency, DRAM provides the required capacity and bandwidth, and MRAM provides non-volatility with fast access. A **SRAM** → **DRAM** → **MRAM** hierarchy provides a good balance between latency, bandwidth, capacity, and power.

#### 4. AI Inference Edge Device

**An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.**

#### Problem Understanding

An edge AI device must load machine-learning models quickly while consuming minimum power. The memory system must provide large persistent storage for models, fast working memory for inference, and very fast access to frequently used data.

#### NAND Flash vs DRAM vs RRAM

| Feature    | NAND Flash    | DRAM         | RRAM                     |
|------------|---------------|--------------|--------------------------|
| Volatility | Non-volatile  | Volatile     | Non-volatile             |
| Speed      | Moderate      | Very high    | High                     |
| Capacity   | High          | Medium/High  | Potentially high         |
| Power      | Low standby   | Higher       | Low                      |
| Main use   | Model storage | Runtime data | Fast non-volatile memory |

#### Proposed Memory Hierarchy

**AI Accelerator/CPU → SRAM Cache → DRAM → RRAM → NAND Flash**

## Operation

- **SRAM:** Stores frequently accessed weights, instructions, and intermediate values close to the processor.
- **DRAM:** Holds active neural-network models, tensors, activations, and temporary inference data.
- **RRAM:** Stores frequently used model parameters or data that benefits from fast non-volatile access.
- **NAND Flash:** Stores complete AI models, operating-system files, applications, and large datasets.

## Optimization Techniques

1. Store frequently used models in faster memory.
2. Use model quantization to reduce memory requirements.
3. Compress model weights where possible.
4. Keep active inference tensors in DRAM.
5. Use SRAM for frequently reused data.
6. Use DMA to transfer data efficiently.
7. Move rarely used models to NAND Flash.

## Solution

### Comparison of Memory Technologies

#### 1. NAND Flash

- Non-volatile memory.
- Large storage capacity.
- Lowest cost per GB.
- Slowest access speed.

**Use:** Stores AI models, datasets, and operating system files.

#### 2. DRAM

- Volatile memory.
- High speed and high bandwidth.
- Moderate power consumption.

**Use:** Loads and executes AI models during inference.

#### 3. RRAM (Resistive RAM)

- Non-volatile memory.
- Faster than NAND Flash.
- Lower power consumption than DRAM.
- Retains data without power.

**Use:** Stores frequently used AI models and inference parameters.

## OutCome:

- Faster AI model loading
- Reduced inference latency
- Lower power consumption

- Faster startup time
- Improved battery life for edge devices

## Conclusion

The optimized hierarchy is:

**SRAM → DRAM → RRAM → NAND Flash**

This provides fast inference while keeping large model storage on low-cost, non-volatile NAND Flash. RRAM can further reduce model-loading latency and standby power in suitable edge architectures.

## 5. High-Performance Gaming Console

**A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.**

### Problem Understanding

A gaming console experiences frame drops while loading large scenes. Modern games continuously stream textures, geometry, shaders, audio, and other assets. Insufficient memory bandwidth or inefficient cache usage can delay the GPU and cause frame-time spikes.

### L3 Cache vs DRAM

| Feature   | L3 Cache               | DRAM              |
|-----------|------------------------|-------------------|
| Capacity  | Small                  | Large             |
| Latency   | Very low               | Higher            |
| Bandwidth | High for local access  | Very high overall |
| Location  | Close to CPU           | Main memory       |
| Best use  | Frequently reused data | Large game assets |

L3 cache is useful for frequently accessed data because it has much lower latency than DRAM. However, large textures, geometry, and game assets cannot all fit into cache. They must be stored in DRAM.

### Proposed Memory Hierarchy

**CPU/GPU → L1/L2 Cache → L3 Cache → High-Bandwidth DRAM → NVMe SSD**

### Improvements

1. Increase L3 cache capacity where feasible.
2. Use high-bandwidth GDDR-class memory for GPU workloads.
3. Use multiple memory channels to increase throughput.
4. Use fast NVMe storage for game asset streaming.
5. Apply texture and asset compression.
6. Use prefetching to load assets before they are needed.
7. Use DMA for efficient data movement.
8. Optimize game-engine memory allocation.
9. Keep frequently reused textures and geometry in cache.

## 10. Reduce unnecessary CPU-GPU data transfers.

### Analysis

Increasing L3 cache helps reduce latency for repeatedly accessed data, but it cannot solve the entire problem because large scenes require substantial memory capacity and bandwidth. Therefore, improving DRAM bandwidth and asset-streaming performance is also essential.

### Conclusion

The most effective design is a **multi-level cache + high-bandwidth DRAM + fast NVMe storage** architecture. Intelligent prefetching and compression can further reduce frame drops during large-scene loading.

### Code of Conduct:

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

### MARKS ALLOCATION

|    | Understanding of Industry Problem (20%) | Application of Engineering Concepts (25%) | Solution Design (25%) | Use of Tools (15%) | Analysis (10%) |
|----|-----------------------------------------|-------------------------------------------|-----------------------|--------------------|----------------|
| Q1 |                                         |                                           |                       |                    |                |
| Q2 |                                         |                                           |                       |                    |                |
| Q3 |                                         |                                           |                       |                    |                |
| Q4 |                                         |                                           |                       |                    |                |
| Q5 |                                         |                                           |                       |                    |                |

### RUBRICS FOR CALCULATION:

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|----------|-----------|---------------------|----------------------|----------------------|---------------------|
|----------|-----------|---------------------|----------------------|----------------------|---------------------|

|                                     |     |                                                                                    |                                                      |                                                |                                                   |
|-------------------------------------|-----|------------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| Understanding of Industry Problem   | 20% | Comprehensive understanding of real-world problem and constraints(1)               | Good understanding with minor gaps in requirements   | Basic understanding; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25% | Applies advanced and relevant concepts effectively to solve the problem(1.25)      | Applies appropriate concepts with minor errors       | Limited application of concepts                | Incorrect or no application of concepts           |
| Solution Design                     | 25% | Innovative, practical, and feasible solution aligned with industry standards(1.25) | Logical and feasible solution with minor gaps        | Basic solution with limited feasibility        | Unclear or impractical solution                   |
| Use of Tools                        | 15% | Effective use of modern tools, technologies, and real data(0.75)                   | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools           | Minimal or incorrect use of tools                 |
| Analysis                            | 10% | Thorough analysis with validated results and performance evaluation(0.5)           | Good analysis with reasonable validation             | Basic analysis with limited validation         | Poor or no analysis                               |